Testing Report 1: Deliverable 3 – Security Testing and Analysis

Secure Real Time Chat Application

Report Date: May 5, 2026

Executive Summary

This report presents the security testing and analysis carried out for Deliverable 3 of the Secure Software Development course. The system tested is a full stack real time chat application developed using FastAPI for the backend and Next.js for the frontend.

The testing process focused on evaluating the security posture of the backend application, ensuring that authentication, authorization, input validation, and database interactions are implemented correctly and securely.

A total of nine security focused tests were developed and executed, and all of them passed successfully. No critical, high, or medium severity vulnerabilities were identified. A few low level issues were detected, but they were limited to test data and do not affect production security.

Overall, the application demonstrates strong security practices and is considered stable and secure for deployment, with only minor improvements recommended.

1. Testing Objectives

The main goal of this testing process was to evaluate the security of the application and verify that best practices were followed during development.

The key objectives included identifying possible vulnerabilities, validating authentication and authorization mechanisms, checking input validation, and assessing code quality and maintainability.

The testing scope covered the backend application, including authentication, authorization, database access, and cryptographic handling. The focus remained on unit level and security focused testing rather than full system or performance testing.

Frontend security, infrastructure level concerns, and load testing were not included in this phase.

2. Testing Methodology

A structured approach was followed to ensure proper coverage and organization of the testing process.

The process started with setting up the environment, followed by writing unit tests specifically targeting security aspects. After that, static security analysis was performed, along with code quality evaluation. Finally, results were documented and submitted.

Several tools were used during testing. Pytest was used to create and run unit tests. Bandit was used for static security analysis to detect vulnerabilities in the code. Radon helped measure code complexity and maintainability. Coverage tools were used to evaluate how much of the codebase was tested.

The testing environment was set up on Windows using Python in a virtual environment. SQLite was used as a temporary database for testing purposes.

3. Testing Execution and Results

Nine unit tests were created focusing on authentication and security scenarios. These tests included validating endpoints, handling missing inputs, rejecting invalid credentials, preventing SQL injection, and ensuring protected routes require valid tokens.

All tests passed successfully, indicating that the implemented security mechanisms are functioning correctly.

Code coverage reached around half of the total codebase. While this is acceptable for a security focused test suite, there is room to expand coverage in future work.

Static security testing did not reveal any serious vulnerabilities. All detected issues were low severity and related to test code rather than production logic. No unsafe coding practices such as hardcoded secrets, insecure cryptography, or dangerous functions were found in the main application.

Code quality analysis showed excellent results, with most functions having very low complexity. A few functions had moderate complexity due to their critical roles, but this was considered acceptable.

4. Security Findings

The application demonstrates strong implementation of authentication and authorization. Passwords are securely hashed, tokens are properly generated and validated, and protected endpoints enforce access control correctly.

Input validation is handled well through structured models, ensuring that invalid or malicious data is rejected before processing. SQL injection attempts were tested and successfully blocked.

Database interactions are secure, with proper use of parameterized queries and ORM features. No unsafe query patterns were found.

Error handling is implemented carefully, ensuring that users do not receive sensitive information while detailed logs are maintained internally.

One minor issue was identified in the form of a hardcoded file path used for logging. While this does not create a security risk, it reduces portability and should be corrected.

5. Vulnerabilities and Improvements

No critical or high level vulnerabilities were found during testing. The only issue identified was a hardcoded log file path, which can cause problems when the application is moved to a different environment.

It is recommended to replace this with a dynamic path that adapts to the project structure.

For production deployment, additional improvements such as enabling HTTPS, configuring proper security headers, and restricting allowed origins should be implemented.

6. Recommendations

Although the application is secure, a few improvements can make it even better.

The logging configuration should be updated to avoid hardcoded paths. Security headers such as content security policy and transport security should be added before deployment. CORS settings should be restricted to the production domain.

Future testing can include expanding test coverage, adding integration tests, and performing manual penetration testing.

7. Conclusion

The results of this testing process show that the application has a strong security foundation. All major security mechanisms are properly implemented, and no serious vulnerabilities were found.

The code is clean, maintainable, and well organized. Authentication, authorization, input validation, and database security have all been handled effectively.

With a few minor adjustments for production readiness, the application can be confidently deployed.